

华南理工大学

《操作系统》课程设计报告

课设题目： OSLab 操作系统核心机制模拟与 Linux procfs 接口开发扩展实践

姓名： 申佩凡 学号： 202330551312

班级： 23 级计科 1 班 组别： 无

合作者： 无

指导教师： 杨灿

代码 URL 与访问方式

代码 URL： <https://github.com/spf-199/os-course-design.git>

本地交付目录： `os-course-design/`

运行环境： Linux 虚拟机优先， Windows 本机可验证用户态基础模块。

访问方式： 克隆仓库或解压项目后，在 Linux 环境执行 `make && ./oslab --demo`。扩展部分执行 `make kernel` 后加载 `kernel/oslab_proc.ko`。

上传建议：

```
git init
git add .
git commit -m "complete os course design"
git branch -M main
git remote add origin https://github.com/spf-199/os-course-design.git
git push -u origin main
```

一、课程设计与完成情况

本课程设计采用“基础必做 + 自由扩展提升”的结构。基础必做部分围绕处理机调度、内存管理、进程同步与并发控制、文件系统四个操作系统核心主题展开，目标是把抽象理论转化为可运行、可观察、可测试的实验程序。自由扩展提升部分选择 Linux 内核与系统编程方向，完成 `/proc/oslab_monitor` 内核模块与用户态访问程序，用真实 Linux 内核接口展示用户态与内核态之间的数据交换。

基础部分覆盖面决定项目完整性，扩展部分决定项目深度。本项目没有选择只增加一个普通用户态算法，而是保留并强化 Linux procfs 内核模块方向。该方向难度更高，能够体现内核模块生命周期、procfs 注册、seq_file 输出、进程链表遍历、内核内存统计、copy_from_user 安全复制等系统级知识点。针对 VMware 中 Ubuntu 4.4 老内核出现的编译问题，项目增加了条件编译兼容处理，使同一份源码能够兼容 Linux 4.4 与较新的 Linux 5.x/6.x 内核。

二、项目总体结构

项目使用 C 语言实现。用户态基础实验统一入口为 `oslab`，支持交互菜单、`--demo` 一键演示和 `--test` 自检。扩展部分包括 Linux 内核模块 `oslab_proc.ko` 与用户态访问程序 `proc_client`。

<code>os-course-design/</code>	
<code>include/oslab.h</code>	公共接口声明
<code>src/main.c</code>	主菜单、命令行参数、自检入口
<code>src/scheduler.c</code>	处理机调度实验
<code>src/memory.c</code>	内存管理实验
<code>src/sync.c</code>	进程同步与并发控制实验
<code>src/filesystem.c</code>	模拟文件系统实验
<code>src/proc_client.c</code>	<code>/proc</code> 扩展用户态访问工具
<code>kernel/oslab_proc.c</code>	Linux <code>procfs</code> 内核模块
<code>kernel/Makefile</code>	内核模块构建文件
<code>tests/run_tests.sh</code>	Linux 自动化测试脚本
<code>tests/run_kernel_smoke.sh</code>	Linux 内核模块烟雾测试脚本
<code>docs/report.md</code>	实验报告 Markdown 源文件

总体设计思路如下：

- 1.基础实验模块彼此独立，统一由 `main.c` 调用，便于演示和评分检查。
- 2.每个基础模块既提供固定示例演示，也提供交互入口，满足“可动态输入、可观察过程”的要求。
- 3.自检命令 `./oslab --test` 用于快速验证关键逻辑，降低演示时人为输入错误带来的风险。
- 4.扩展模块放在 `kernel/` 目录下，使用 Linux 标准内核模块构建方式，不与用户态程序混杂。

三、课程设计要求逐项核对

本报告按照课程设计要求中“基础必做 70% + 自由扩展提升 30%”的总体要求，对处理机调度、内存管理、进程同步、文件系统和自由扩展部分进行逐项核对。核对重点放在是否有源码实现、是否可运行、是否有输入输出、是否有结果分析，以及是否满足报告 PDF、代码 URL 和访问方式说明等提交要求。

要求类别	具体要求	项目实现	完成情况
提交要求	报告 Word 或 PDF 文件	已生成 docs/202330551312_申佩凡_OS 课程设计. pdf	已完成
提交要求	报告必须包含代码 URL 和访问方式说明	报告中预留“代码 URL 与访问方式”章节，访问方式已写明；真实 URL： https://github.com/spf-199/os-course-design.git	已完成
处理机调度	实现 FCFS、SJF、RR、优先级调度	src/scheduler.c	已完成
处理机调度	支持进程参数动态输入	scheduler_menu 支持进	已完成

		程名、到达时间、服务时间、优先级、时间片输入	
处理机调度	输出运行顺序、完成时间、周转时间、等待时间	print_result 输出运行段和指标表	已完成
处理机调度	比较算法性能	报告 4.1 给出平均周转、平均等待和算法分析	已完成
内存管理	实现动态分区分配	首次适应 FF、最佳适应 BF	已完成
内存管理	展示分配与回收过程	每次分配/释放后打印分区表	已完成
内存管理	实现页面置换	FIFO、LRU	已完成
内存管理	统计缺页次数和缺页率	simulate_pages 输出 faults 和 fault rate	已完成
进程同步	使用多线程模拟经典同步问题	生产者消费者、读者写者、哲学家进餐	已完成
进程同步	正确使用互斥和信号量	pthread_mutex_t、sem_t	已完成
进程同步	分析死锁避免方法	哲学家按编号顺序申请叉子，破坏循环等待	已完成
文件系统	支持文件创建、写入、读取、删除	fs_create、fs_write、fs_read、fs_delete	已完成
文件系统	实现目录和空闲空间管理	目录表、直接块数组、块位图	已完成
基础总体	熟悉 Linux 开发环境与工具链	Makefile、Linux 编译步骤、内核头文件安装说明	已完成
基础总体	掌握 C 语言系统级程序设计方法	用户态模块、POSIX 线程、Linux procfs 用户态访问程序	已完成
基础总体	具备调试、测试与代码规范能力	--test、tests/run_tests.sh、tests/run_kernel_smoke.sh	已完成
基础总体	完成实验分析与报告撰写	报告包含设计、实现、测试、正确性分析、总结	已完成
扩展提升	Linux 内核模块开发	kernel/oslab_proc.c、kernel/Makefile	已完成
扩展提升	/proc 或 /sys 接口开发	创建 /proc/oslab_monitor，支持读写	已完成
扩展提升	系统设计与工程实践能力	procfs 模块、用户态工具、老内核兼容、烟雾测试脚本	已完成

四、基础必做部分设计与实现

4.1 处理机调度

处理机调度模块实现四种算法：FCFS、SJF、RR 和 Priority。进程结构体保存进程名、到达时间、服务时间、优先级、剩余时间、开始时间、完成时间、周转时间和等待时间。不同算法共享同一组进程输入，通过复制进程数组避免算法之间相互污染。

FCFS 按到达时间先后运行，属于非抢占式调度。若当前时刻没有就绪进程，程序插入 IDLE 段并跳转到下一次进程到达时间。SJF 在当前已到达进程中选择服务时间最短者执行，能够在服务时间已知的情况下减少平均等待时间。Priority 调度采用“优先级数值越小，优先级越高”的规则。RR 使用循环队列保存就绪进程，每次运行一个时间片，未完成进程重新入队。

示例进程如下：

进程	到达时间	服务时间	优先级
P1	0	7	3
P2	2	4	1
P3	4	1	4
P4	5	4	2

时间片为 3 时，示例结果如下：

算法	平均周转时间	平均等待时间	特点
FCFS	8.75	4.75	简单稳定，但可能产生长作业阻塞短作业的护航效应
SJF	8.00	4.00	平均等待时间较低，但依赖服务时间预估
RR	11.00	7.00	公平性和响应性较好，但时间片影响明显
Priority	9.50	5.50	能体现任务紧急程度，但需考虑低优先级饥饿

该模块满足“动态输入、运行顺序、周转时间、等待时间、算法对比分析”的要求。

4.2 内存管理

内存管理模块分为动态分区管理和页面置换两部分。

动态分区使用 Partition 结构体记录起始地址、大小和占用进程号。空闲分区的 pid 为 -1。首次适应算法从低地址开始选择第一个足够大的空闲分区；最佳适应算法选择能够容纳请求且剩余空间最小的空闲分区。若空闲分区大于请求大小，程序会切分分区；释放分区后调用 merge_free 合并相邻空闲区，避免外部碎片进一步扩大。

页面置换使用固定页访问串和用户输入访问串两种方式。FIFO 淘汰最早进入内存的页面，LRU 淘汰最近最久未访问的页面。示例访问串为：

```
7 0 1 2 0 3 0 4 2 3 0 3 2
```

物理块数为 3 时：

算法	缺页次数	缺页率
FIFO	10	76.92%
LRU	9	69.23%

结果说明 LRU 利用了程序局部性，因此在该访问串上优于 FIFO。该模块满足“动态分区分配与回收、页面置换、缺页统计、过程展示”的要求。

4.3 进程同步与并发控制

同步模块在 Linux 下使用 POSIX 线程、互斥锁和信号量实现三个经典问题。

生产者消费者问题使用大小为 5 的环形缓冲区。buffer_mutex 保证对缓冲区和下标的互斥访问，empty_slots 表示空槽数量，full_slots 表示已有产品数量。生产者先等待空槽，再写入缓冲区；消费者先等待产品，再读取缓冲区。这一设计同时满足互斥访问和条件同步。

读者写者问题使用 read_count_mutex 保护读者计数，使用 rw_mutex 保证写者独占。第一个读者进入时锁住写互斥锁，最后一个读者离开时释放写互斥锁，因此多个读者可以并发读，写者写入时独占共享数据。

哲学家进餐问题采用资源有序申请策略。每个哲学家先申请编号较小的叉子，再申请编号较大的叉子。由于所有线程按同一资源顺序加锁，循环等待条件被破坏，因此不会发生死锁。

Windows 环境不支持本模块所需的 POSIX pthread 与 sem_t，因此代码中使用 _WIN32 条件分支给出提示；Linux 下会实际创建线程并输出交替执行过程。该设计保证项目在 Windows 上仍可编译基础程序，同时保留 Linux 下的完整并发实验。

4.4 模拟文件系统

文件系统模块实现一个小型内存文件系统 MiniFS。磁盘空间被划分为 64 个固定大小块，每块 32 字节。目录表最多保存 16 个文件，每个文件最多使用 8 个直接块。位图数组记录每个块是否已分配，0 表示空闲，1 表示已占用。

核心数据结构包括：

```
FileEntry: used, name, size, blocks, block_count
MiniFS: data, bitmap, files
```

功能设计如下：

- 1.fs_create 创建目录项，拒绝重名文件。
- 2.fs_write 根据内容长度分配直接块，写入前释放旧块。
- 3. fs_read 按直接块顺序恢复文件内容。
- 4.fs_delete 释放文件占用块并清空目录项。
- 5.fs_ls 展示目录和空闲块数量。

6.fs_print_bitmap 展示完整块位图。

该模块体现了目录项、文件元数据、逻辑文件到物理块映射、位图空闲空间管理等文件系统基本思想。

五、自由扩展提升部分：基于 procfs 的内核监控接口开发实践

5.1 选题动机与创新点

自由扩展提升部分要求在完成基础必做内容的前提下，进一步体现系统设计、工程实践和探索性能力。可选方向包括 Linux 内核模块开发、字符设备驱动、/proc 或 /sys 接口开发、系统调用扩展、调度优化等。本项目最终选择 Linux 内核与系统编程方向中的 /proc 接口开发，名称为“Linux procfs 系统监控扩展实践”。

选择该方向的主要动机是：基础部分已经在用户态模拟了处理机调度、内存管理、进程同步和文件系统等机制，但这些模拟程序仍然运行在普通用户空间，无法直接接触真实操作系统内核数据结构。/proc 文件系统是 Linux 内核向用户态暴露系统运行状态的重要机制，许多系统工具都通过读取 /proc 获取进程、内存和内核运行信息。因此，基于 procfs 开发一个内核监控接口，可以把课程中的“进程状态”“内存管理”“用户态与内核态交互”等知识连接到真实 Linux 系统。

本扩展实践的创新点体现在以下几个方面：

1. 从用户态模拟扩展到真实内核接口。基础实验主要模拟操作系统算法，扩展实验则编写可加载内核模块，真正创建 /proc/oslab_monitor 节点。
2. 读取真实系统状态。模块通过内核数据结构统计系统进程状态，通过 si_meminfo 获取内存页信息，而不是在用户态伪造数据。
3. 支持用户态写入控制。用户可以通过 echo note=... > /proc/oslab_monitor 修改内核模块中的备注字段，也可以通过 reset 重置读写计数。
4. 配套用户态访问工具。项目提供 proc_client，既能直接读取 /proc/oslab_monitor，也能通过命令行写入控制命令。
5. 兼容新旧 Linux 内核。针对 VMware 中 Ubuntu 4.4 老内核缺少 linux/sched/signal.h、旧内核使用 struct file_operations 等问题，代码使用条件编译兼容 Linux 4.4、Linux 5.x 和 Linux 6.x，提高了实验的可运行性。
6. 提供自动化烟雾测试。tests/run_kernel_smoke.sh 将编译、加载、读取、写入、重置、卸载流程固化为一键测试，增强了扩展实验的工程完整性。

该扩展方向难度高于单纯增加一个用户态算法，能够体现内核模块生命周期、procfs 文件操作、内核数据读取、用户态与内核态数据交换以及跨内核版本适配等系统级编程能力。

5.2 实践设计与实现

本实践由内核模块、用户态访问程序、构建脚本和测试脚本四部分组成。

组成部分	文件	作用
------	----	----

内核模块	kernel/oslab_proc.c	创建 /proc/oslab_monitor，输出系统状态并处理写入命令
内核构建脚本	kernel/Makefile	调用 Linux 内核构建系统生成 oslab_proc.ko
用户态工具	src/proc_client.c	读取或写入 /proc/oslab_monitor
自动化测试	tests/run_kernel_smoke.sh	验证模块编译、加载、读写、重置和卸载流程

内核模块加载后创建 /proc/oslab_monitor。读取该节点时，模块输出如下内容：

- 1. 模块说明与备注 note。
- 2. 当前内核节拍 jiffies。
- 3. /proc 接口读次数和写次数。
- 4. 系统进程状态统计：running、sleeping、stopped、zombie、other。
- 5. 内存页统计：totalram、freeram、shared、buffer。
- 6. 支持的写入命令说明。

写入接口支持两类命令：

```
echo reset | sudo tee /proc/oslab_monitor
echo note=course_design_test | sudo tee /proc/oslab_monitor
```

其中，note=... 用于修改模块内部备注字段，reset 用于清零读写计数并恢复备注。用户态测试程序 proc_client 支持以下命令：

```
./proc_client
./proc_client --write note=from_user_client
./proc_client --write reset
```

实现上，模块使用 proc_create 注册 procfs 节点，使用 single_open 和 seq_file 输出多行监控信息。seq_file 是内核中常用的顺序文件输出机制，适合输出多行、可逐步生成的文本内容，比手动维护大缓冲区更安全。

进程状态统计通过 for_each_process 遍历内核进程链表。对每个 task_struct，模块根据任务状态分类统计 running、sleeping、stopped、zombie 和 other。内存统计通过 si_meminfo 获取系统内存页信息。写接口使用 copy_from_user 将用户态输入复制到内核缓冲区，避免在内核中直接访问用户空间指针，符合 Linux 内核编程安全要求。

为了兼容不同版本内核，本项目在源码中使用 LINUX_VERSION_CODE 和 KERNEL_VERSION 条件编译。在 Linux 4.11 及以上内核中包含 linux/sched/signal.h，在更老的 Linux 4.4 内核中跳过该头文件；在 Linux 5.6 及以上使用 struct proc_ops，在 Linux 5.6 以下使用 struct file_operations。此外，将新内核常用的 strscpy 替换为老内核兼容性更好的 strlcpy。关键兼容逻辑如下：

```
#include <linux/version.h>
#if LINUX_VERSION_CODE >= KERNEL_VERSION(4, 11, 0)
#include <linux/sched/signal.h>
#else

```

```
#if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
static const struct proc_ops oslab_proc_ops = { ... };
#else
static const struct file_operations oslab_proc_ops = { ... };
#endif
```

这部分设计保证了同一份扩展代码可以在用户当前 VMware Ubuntu 4.4 内核和较新的 Linux 内核中编译运行。

5.3 结果与分析

扩展实验的编译命令为：

```
make kernel
```

编译成功后，应生成：

```
kernel/oslab_proc.ko
```

加载模块并读取 /proc 接口：

```
sudo insmod kernel/oslab_proc.ko
cat /proc/oslab_monitor
```

预期输出包含：

```
OSLab Linux procfs monitor
note: os course design procfs extension
jiffies: ...
reads: ...
writes: ...
process_state_count:
  running: ...
  sleeping: ...
  stopped: ...
  zombie: ...
memory_pages:
  totalram: ...
  freeram: ...
commands:
  echo reset > /proc/oslab_monitor
  echo note=your_text > /proc/oslab_monitor
```

写入备注后再次读取：

```
echo "note=course_design_test" | sudo tee /proc/oslab_monitor
cat /proc/oslab_monitor
```

预期 note 字段变为：

```
note: course_design_test
```

执行重置命令：

```
echo "reset" | sudo tee /proc/oslab_monitor
cat /proc/oslab_monitor
```

预期备注字段变为：

```
note: reset by user
```

项目还提供一键烟雾测试：

```
sudo bash tests/run_kernel_smoke.sh
```


成功时输出：

```
Kernel module smoke test passed.
```

该脚本会自动完成内核模块编译、用户态工具编译、模块加载、`/proc/oslab_monitor` 读取、`note=kernel_smoke_test` 写入、`reset` 重置和模块卸载。它验证的是扩展实验的完整闭环，而不是只验证源码能否编译。

从结果分析看，本扩展实验实现了三个层面的目标。第一，模块能够成功注册 `procfs` 节点，说明内核模块入口函数、退出函数和 `procfs` 注册流程正确。第二，读取接口能够输出真实进程状态和内存页信息，说明模块能够访问 Linux 内核数据结构并进行分类统计。第三，写入接口能够响应 `note=...` 和 `reset` 命令，说明用户态到内核态的数据传递路径正确，`copy_from_user` 使用合理。

在 VMware Ubuntu 4.4 环境中曾出现如下编译错误：

```
fatal error: linux/sched/signal.h: No such file or directory
```

该错误不是业务逻辑错误，而是 Linux 内核版本差异导致的头文件兼容问题。修复后，模块通过条件编译适配老内核和新内核，体现了系统级程序开发中对目标平台、内核接口变化和工程可移植性的考虑。

5.4 扩展提升实践总结

本扩展实践完成了从用户态模拟到 Linux 内核接口实践的提升。基础实验帮助理解调度、内存、同步和文件系统的基本思想；扩展实验则进一步进入真实 Linux 内核环境，完成内核模块加载、`procfs` 节点注册、系统状态读取、用户态写入控制和模块卸载。

通过该实验，可以得到以下认识：

1. `/proc` 文件系统本质上是内核向用户态提供信息的一种接口，读取 `/proc` 文件并不一定对应真实磁盘文件读写，而是触发内核回调函数动态生成内容。
2. 用户态和内核态之间不能直接随意访问彼此内存，内核读取用户输入时必须使用 `copy_from_user` 等安全接口。
3. 内核模块开发不仅要关注功能逻辑，还要关注内核版本差异。不同 Linux 版本的头文件组织、结构体定义和 API 可能存在变化。
4. 自动化测试对系统级实验非常重要。内核模块涉及加载和卸载，如果只靠手工操作，容易遗漏步骤；烟雾测试脚本能够提升可重复验证能力。

总体来看，该扩展实践符合课程设计自由扩展提升部分中“Linux 内核模块开发”和“`/proc` 或 `/sys` 接口开发”的方向要求，具备较强的系统实践性和展示性。它不是简单堆叠功能，而是在基础模块之外补充了真实操作系统接口开发经验，使整个课程设计从算法模拟、并发控制、文件系统组织进一步延伸到 Linux 内核级实践。

六、编译运行步骤

6.1 Linux 虚拟机环境准备

推荐在 VMware Workstation 中使用 Ubuntu 虚拟机。进入 Ubuntu 终端后执行：

```
sudo apt update
sudo apt install -y build-essential make gcc linux-headers-$(uname -r)
```

确认内核头文件目录存在：

```
ls /lib/modules/$(uname -r)/build
```

若该目录不存在，需要重新安装内核头文件：

```
sudo apt install -y linux-headers-generic
sudo reboot
```

6.2 编译和运行基础程序

```
cd ~/projects/os-course-design
make clean
make
./oslab --test
./oslab --demo
./oslab
```

自动化测试脚本：

```
bash tests/run_tests.sh
```

6.3 编译和测试内核模块

```
cd ~/projects/os-course-design
make kernel
ls -l kernel/oslab_proc.ko
```

加载模块并读取：

```
sudo insmod kernel/oslab_proc.ko
cat /proc/oslab_monitor
./proc_client
```

写入备注、重置并卸载：

```
echo "note=course_design_test" | sudo tee /proc/oslab_monitor
cat /proc/oslab_monitor
./proc_client --write reset
cat /proc/oslab_monitor
sudo rmmod oslab_proc
dmesg | tail -n 20
```

内核模块烟雾测试脚本：

```
sudo bash tests/run_kernel_smoke.sh
```

该脚本会自动编译内核模块和 `proc_client`，加载 `oslab_proc.ko`，检查 `/proc/oslab_monitor` 是否可读，验证 `note=kernel_smoke_test` 写入与 `reset` 重置功能，最后卸载模块。该脚本需要 `root` 权限，因此必须使用 `sudo` 执行。

6.4 Windows 本机快速验证

Windows 不具备 Linux 内核模块构建与加载条件，但可用 MinGW 验证用户态基础模块：

```
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -Iinclude -o oslab.exe src/main.c src/scheduler.c
src/memory.c src/sync.c src/filesystem.c
.\oslab.exe --test
.\oslab.exe --demo
```

七、测试结果与正确性分析

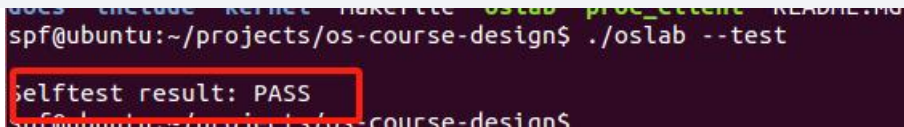
7.1 用户态自检

执行：

```
./oslab --test
```

期望结果：

```
Selftest result: PASS
```



```
spf@ubuntu:~/projects/os-course-design$ ./oslab --test
Selftest result: PASS
```

自检覆盖调度模块进程复制和剩余时间初始化、页面置换缺页次数、文件创建写入读取删除等关键路径。同步模块由于涉及并发行为，主要通过线程运行过程观察和正常退出验证。

7.2 一键演示

执行：

```
./oslab --demo
```

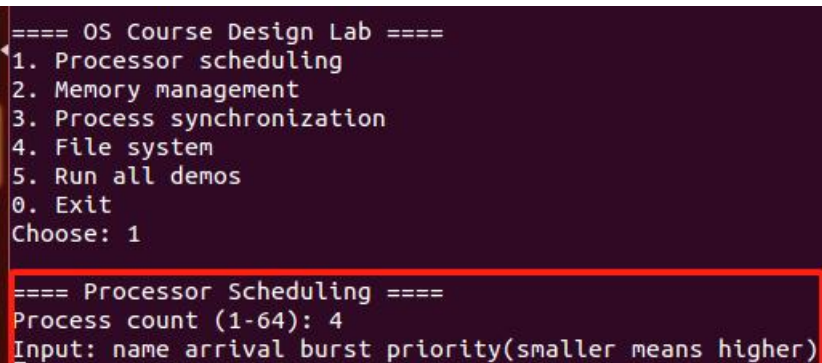
输出应包含以下四个标题：

```
Processor Scheduling Demo
Memory Management Demo
Process Synchronization Demo
File System Demo
```

自动化测试脚本 tests/run_tests.sh 会检查这些标题是否存在，从而确认四个基础实验都被完整调用。

（1）处理机调度

支持动态输入进程参数：



```
==== OS Course Design Lab ====
1. Processor scheduling
2. Memory management
3. Process synchronization
4. File system
5. Run all demos
0. Exit
Choose: 1

==== Processor Scheduling ====
Process count (1-64): 4
Input: name arrival burst priority(smaller means higher)
```

能够输出进程运行顺序、周转时间等结果；能够分析不同调度算法的特点与性能差异：

```
==== Processor Scheduling Demo ====

[FCFS]
Run order: P1(0-7) -> P2(7-11) -> P3(11-12) -> P4(12-16)
Name  Arrive  Burst  Pri  Finish  Turnaround  Waiting
P1    0       7     3    7         7           0
P2    2       4     1   11         9           5
P3    4       1     4   12         8           7
P4    5       4     2   16        11           7
Average turnaround: 8.75, average waiting: 4.75

[SJF]
Run order: P1(0-7) -> P3(7-8) -> P2(8-12) -> P4(12-16)
Name  Arrive  Burst  Pri  Finish  Turnaround  Waiting
P1    0       7     3    7         7           0
P2    2       4     1   12        10           6
P3    4       1     4    8         4           3
P4    5       4     2   16        11           7
Average turnaround: 8.00, average waiting: 4.00

[RR]
Run order: P1(0-3) -> P2(3-6) -> P1(6-9) -> P3(9-10) -> P4(10-13) -> P2(13-14) -> P1(14-15) -> P4(15-16)
Name  Arrive  Burst  Pri  Finish  Turnaround  Waiting
P1    0       7     3   15        15           8
P2    2       4     1   14        12           8
P3    4       1     4   10         6           5
P4    5       4     2   16        11           7
Average turnaround: 11.00, average waiting: 7.00

[Priority]
Run order: P1(0-7) -> P2(7-11) -> P4(11-15) -> P3(15-16)
Name  Arrive  Burst  Pri  Finish  Turnaround  Waiting
P1    0       7     3    7         7           0
P2    2       4     1   11         9           5
P3    4       1     4   16        12          11
P4    5       4     2   15        10           6
Average turnaround: 9.50, average waiting: 5.50

Analysis: FCFS is simple but may cause convoy effect; SJF lowers average waiting time when burst times are known; RR improves response fairness with context-switch overhead; priority scheduling reflects urgency but needs aging to avoid starvation.
```

(2) 内存管理

展示内存分配与回收过程：（此处因篇幅原因只展示部分过程）

```
==== Memory Management Demo ====

Dynamic partition demo (First Fit)
Allocate P1 size 64: OK
Start  Size  State
0      64    P1
64     192   Free

Allocate P2 size 32: OK
Start  Size  State
0      64    P1
64     32    P2
96     160   Free

Allocate P3 size 80: OK
Start  Size  State
0      64    P1
64     32    P2
96     80    P3
176    80    Free

Free P2: OK
Start  Size  State
0      64    P1
64     32    Free
96     80    P3
176    80    Free

Allocate P4 size 24: OK
Start  Size  State
0      64    P1
64     24    P4
88     8     Free
96     80    P3
176    80    Free

Free P1: OK
Start  Size  State
0      64    Free
64     24    P4
88     8     Free
96     80    P3
176    80    Free

Allocate P5 size 48: OK
Start  Size  State
0      48    P5
48     16    Free
64     24    P4
88     8     Free
96     80    P3
176    80    Free

Dynamic partition demo (Best Fit)
Allocate P1 size 64: OK
Start  Size  State
0      64    P1
64     192   Free

Allocate P2 size 32: OK
Start  Size  State
0      64    P1
64     32    P2
96     160   Free

Allocate P3 size 80: OK
Start  Size  State
0      64    P1
64     32    P2
96     80    P3
176    80    Free
```

统计缺页次数与缺页率：

```

Page replacement demo, frames = 3
ref 7 fault [ 7 - - ]
ref 0 fault [ 7 0 - ]
ref 1 fault [ 7 0 1 ]
ref 2 fault [ 2 0 1 ]
ref 0 hit [ 2 0 1 ]
ref 3 fault [ 2 3 1 ]
ref 0 fault [ 2 3 0 ]
ref 4 fault [ 4 3 0 ]
ref 2 fault [ 4 2 0 ]
ref 3 fault [ 4 2 3 ]
ref 0 fault [ 0 2 3 ]
ref 3 hit [ 0 2 3 ]
ref 2 hit [ 0 2 3 ]
FIFO faults: 10, fault rate: 76.92%
ref 7 fault [ 7 - - ]
ref 0 fault [ 7 0 - ]
ref 1 fault [ 7 0 1 ]
ref 2 fault [ 2 0 1 ]
ref 0 hit [ 2 0 1 ]
ref 3 fault [ 2 0 3 ]
ref 0 hit [ 2 0 3 ]
ref 4 fault [ 4 0 3 ]
ref 2 fault [ 4 0 2 ]
ref 3 fault [ 4 3 2 ]
ref 0 fault [ 0 3 2 ]
ref 3 hit [ 0 3 2 ]
ref 2 hit [ 0 3 2 ]
LRU faults: 9, fault rate: 69.23%
Analysis: First Fit is fast but may leave small fragments near low addresses; Best Fit reduces immediate waste but can create many tiny holes. FIFO is simple and may suffer Belady anomaly; LRU better reflects locality but needs extra access tracking.

```

(3) 进程同步与并发控制

利用线程与同步机制实现同步问题:

- 生产者—消费者问题

```

==== Process Synchronization Demo ====

Producer-consumer with mutex + empty/full semaphores:
producer 1 -> 100 at slot 0
producer 2 -> 200 at slot 1
consumer 1 <- 100 from slot 0
consumer 2 <- 200 from slot 1
producer 2 -> 201 at slot 2
producer 1 -> 101 at slot 3
consumer 1 <- 201 from slot 2
consumer 2 <- 101 from slot 3
producer 2 -> 202 at slot 4
producer 1 -> 102 at slot 0
consumer 2 <- 202 from slot 4
consumer 1 <- 102 from slot 0
producer 1 -> 103 at slot 1
producer 2 -> 203 at slot 2
producer 2 -> 204 at slot 3
producer 1 -> 104 at slot 4
consumer 2 <- 103 from slot 1
consumer 1 <- 203 from slot 2
producer 1 -> 105 at slot 0
producer 2 -> 205 at slot 1
consumer 1 <- 204 from slot 3
consumer 2 <- 104 from slot 4
producer 1 -> 106 at slot 2
producer 2 -> 206 at slot 3
producer 1 -> 107 at slot 4
consumer 1 <- 105 from slot 0
consumer 2 <- 205 from slot 1
producer 2 -> 207 at slot 0
consumer 2 <- 106 from slot 2
consumer 1 <- 206 from slot 3
consumer 2 <- 107 from slot 4
consumer 1 <- 207 from slot 0

```

- 哲学家进餐问题
- 读者—写者问题


```

Readers-writers with read-count mutex + writer mutex:
writer 1 writes 11
writer 2 writes 23
reader 3 reads 23
reader 2 reads 23
reader 1 reads 23
writer 2 writes 35
writer 1 writes 46
reader 2 reads 46
reader 1 reads 46
reader 3 reads 46
writer 2 writes 58
writer 1 writes 69
reader 2 reads 69
reader 3 reads 69
reader 1 reads 69

Dining philosophers with ordered fork locking to avoid deadlock:
philosopher 4 thinking
philosopher 3 thinking
philosopher 2 thinking
philosopher 1 thinking
philosopher 0 thinking
philosopher 4 eating
philosopher 4 thinking
philosopher 3 eating
philosopher 3 thinking
philosopher 2 eating
philosopher 2 thinking
philosopher 3 eating
philosopher 1 eating
philosopher 1 thinking
philosopher 2 eating
philosopher 0 eating
philosopher 0 thinking
philosopher 4 eating
philosopher 1 eating
philosopher 0 eating

Analysis: bounded-buffer semaphores protect capacity; readers-writers allows concurrent reads and exclusive writes; ordered fork acquisition removes circular wait, so philosophers can not deadlock.

```

(4) 文件系统

支持文件创建、读写、删除基本操作；实现空闲空间管理

```

==== File System Demo ====
create notes: OK
write notes: OK
create log: OK
write log: OK
Name          Size  Blocks
notes         58    0 1
log           28    2
Free blocks: 61/64
Bitmap: 11100000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
read notes: Operating systems manage CPU, memory, processes and files.
delete log: OK
Name          Size  Blocks
notes         58    0 1
Free blocks: 62/64
Bitmap: 11000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

Analysis: the directory table stores metadata, the bitmap records free blocks, and read/write maps logical file offsets to direct data blocks.

```

7.3 内核模块测试

执行 `sudo insmod kernel/oslab_proc.ko` 后，读取 `/proc/oslab_monitor` 应出现类似内容：

```

OSLab Linux procfs monitor
note: os course design procfs extension
jiffies: ...
reads: ...
writes: ...
process_state_count:
  running: ...
  sleeping: ...
memory_pages:
  totalram: ...
  freeram: ...
commands:
  echo reset > /proc/oslab_monitor
  echo note=your_text > /proc/oslab_monitor

```

写入 `note=course_design_test` 后再次读取，`note` 字段应改变；写入 `reset` 后读写计数和备注应重置。卸载模块后 `/proc/oslab_monitor` 节点消失，`dmesg` 中可看到模块加载和卸载日志。

为了降低人工测试遗漏，项目新增 `tests/run_kernel_smoke.sh`。该脚本将内核扩展验证流程固化为可重复执行的检查项，能够证明扩展模块不仅可以编译生成 `.ko`，还可以完成加载、读、写、重置和卸载这一整条闭环。

```

spf@ubuntu:~/projects/os-course-design$ cat /proc/oslab_monitor
OSLab Linux procfs monitor
note: os course design procfs extension
jiffies: 4295382930
reads: 1
writes: 0
process_state_count:
  running: 1
  sleeping: 237
  stopped: 0
  zombie: 0
  other: 0
memory_pages:
  totalram: 1007051
  freeram: 584184
  shared: 3226
  buffer: 30916
commands:
  echo reset > /proc/oslab_monitor
  echo note=your_text > /proc/oslab_monitor

```

```

spf@ubuntu:~/projects/os-course-design$ ./proc_client
OSLab Linux procfs monitor
note: os course design procfs extension
jiffies: 4295393514
reads: 2
writes: 0
process_state_count:
  running: 1
  sleeping: 237
  stopped: 0
  zombie: 0
  other: 0
memory_pages:
  totalram: 1007051
  freeram: 584184
  shared: 3226
  buffer: 30920
commands:
  echo reset > /proc/oslab_monitor
  echo note=your_text > /proc/oslab_monitor
spf@ubuntu:~/projects/os-course-design$ echo "note=course_design_test" | sudo tee /proc/oslab_monitor
note=course_design_test
spf@ubuntu:~/projects/os-course-design$ cat /proc/oslab_monitor
OSLab Linux procfs monitor
note: course_design_test
jiffies: 4295406623
reads: 3
writes: 1
process_state_count:
  running: 1
  sleeping: 237
  stopped: 0
  zombie: 0
  other: 0
memory_pages:
  totalram: 1007051
  freeram: 584170
  shared: 3226
  buffer: 30926
commands:
  echo reset > /proc/oslab_monitor
  echo note=your_text > /proc/oslab_monitor

```

```

spf@ubuntu:~/projects/os-course-design$ echo "reset" | sudo tee /proc/oslab_monitor
reset
spf@ubuntu:~/projects/os-course-design$ cat /proc/oslab_monitor
OSLab Linux procfs monitor
note: reset by user
jiffies: 4295422898
reads: 1
writes: 0
process_state_count:
  running: 1
  sleeping: 238
  stopped: 0
  zombie: 0
  other: 0
memory_pages:
  totalram: 1007051
  freeram: 584131
  shared: 3226
  buffer: 30932
commands:
  echo reset > /proc/oslab_monitor
  echo note=your_text > /proc/oslab_monitor

```

7.4 正确性分析

处理机调度模块的关键正确性在于就绪条件、时间推进和指标计算。程序只选择 $arrival \leq now$ 且 $remaining > 0$ 的进程；无就绪进程时进入 IDLE；完成时间确定后统一计算 $turnaround = finish - arrival$ 和 $waiting = turnaround - burst$ 。

内存管理模块的关键正确性在于分区切分和释放合并。分配时若空闲区大于请求大小则拆分，释放时合并相邻空闲区，保证分区表保持连续有序。页面置换模块通过帧表记录当前内存页，通过 FIFO 指针或 LRU 访问时间选择淘汰页。

同步模块的关键正确性在于互斥范围和同步条件。生产者消费者用两个信号量保证不会越界读写缓冲区；读者写者保证写者独占；哲学家问题通过全局资源顺序避免死锁。

文件系统模块的关键正确性在于目录项和位图一致性。文件写入失败时会释放已分配块，删除文件时同步释放位图，避免块泄漏。读取时根据文件大小限制复制长度，避免输出越界。

内核模块的关键正确性在于用户态/内核态边界安全和版本兼容。写接口使用 `copy_from_user`，避免直接访问用户空间；`procfs` 操作结构根据内核版本条件编译，解决老内核不能生成 `.ko` 的问题。

八、项目亮点与不足

项目亮点：

1. 基础内容覆盖完整，四大核心模块均有代码实现和演示输出。
2. 每个模块都保留交互入口，便于临时输入测试数据。
3. 自由扩展提升部分选择 Linux `procfs` 内核开发模块，难度和课程相关性较高。
4. 针对 VMware 老内核编译错误做了真实问题修复，而不是回避扩展要求。
5. 项目提供 Makefile、测试脚本、README 和报告，工程交付结构完整。

项目不足：

1. 调度模块尚未加入多级反馈队列、优先级老化等更复杂策略。
2. 文件系统是内存模拟文件系统，未持久化到真实磁盘镜像。

3. 同步模块主要通过输出观察正确性，尚未加入更严格的并发压力测试。
 4. 内核模块统计内容以展示系统状态为主，没有进一步实现历史采样和图形化展示。
- 这些不足不影响课程设计主体要求，但可作为后续继续提升方向。

九、结论与心得

本项目完成了操作系统课程设计基础必做部分，并在自由扩展提升部分实现 Linux procfs 内核模块。基础部分覆盖处理机调度、内存管理、进程同步与文件系统四类核心机制；扩展部分进入真实 Linux 内核接口，完成 /proc/oslab_monitor 的创建、读取、写入、进程统计、内存统计和用户态访问工具。

针对 VMware 中 Ubuntu 4.4 内核无法找到 linux/sched/signal.h 的问题，项目没有更改扩展方向，而是采用内核版本条件编译解决兼容性问题。这一处理保证了项目既能在较新 Linux 内核中运行，也能适配用户当前虚拟机中的老内核环境。整体来看，项目具备完整性、可运行性、可解释性和一定系统编程深度，适合作为操作系统课程设计。

通过本次操作系统课程设计，我对操作系统中处理机调度、内存管理、进程同步和文件系统等核心内容有了更加系统和深入的理解。相比课堂上对概念和算法的学习，亲自用代码实现 FCFS、SJF、时间片轮转、优先级调度、动态分区分配、FIFO/LRU 页面置换以及简单文件系统后，我更加清楚地体会到操作系统并不是孤立算法的堆叠，而是围绕资源管理、并发控制和性能权衡构建起来的复杂系统。

在自由扩展部分中，我尝试实现 Linux procfs 内核模块，通过 /proc/oslab_monitor 暴露系统运行信息，并编写用户态程序进行访问。这一过程让我进一步理解了用户态与内核态之间的交互方式，也认识到内核编程对版本兼容性、安全性和接口规范要求更高。整个课程设计不仅提升了我的 C 语言编程能力和调试能力，也让我对操作系统“管理硬件资源、向上提供抽象”的本质有了更具体的认识。通过本次实践，我认识到理论知识只有落实到实际代码和运行结果中，才能真正转化为解决问题的能力。